

iGauchoback — Portfolio Tech Stack & Resume Guide

Candidate portfolio document · FAANG recruiter perspective

Live product: <https://hackathon-nu-taupe.vercel.app>

Repo: github.com/huynghuyen9999/Hackathon (subfolder: `acadmap`)

Executive summary (30-second pitch)

I built **iGauchoback**, a production-deployed full-stack web app that turns official UC degree requirements into interactive prerequisite graphs for **72 majors across 3 campuses**. I owned the entire stack: Next.js App Router frontend, 24 REST API routes, Supabase Postgres with RLS, AI-assisted PDF transcript parsing, edge caching, rate limiting, and observability—shipping from hackathon prototype to hardened Vercel production with real users.

Use this pitch in interviews, LinkedIn About, and the top of your resume project section.

Part 1 — Complete tech stack

Frontend

Layer	Technology	How you used it
Framework	Next.js 14.2 (App Router)	Server Components for data pages; client islands for React Flow, planner, forms
UI library	React 18.3	77+ components, custom design system (no component library)
Language	TypeScript 5.7	Strict typing across API contracts, domain models, Zod schemas
Styling	Tailwind CSS 3.4	Responsive layout, dark mode, brand theming
Graph viz	@xyflow/react 12.3	Interactive degree roadmaps with prerequisite edges, node selection, analysis overlays
Fonts	Inter via <code>next/font</code>	Performance-optimized font loading

Backend & API

Layer	Technology	How you used it
API style	Next.js Route Handlers	24 REST endpoints under <code>app/api/</code>
Runtime	Node.js (Vercel Fluid Compute)	Transcript parsing, admin ops, planner CRUD
Validation	Zod 4.4	AI structured output, request body validation
Middleware	Next.js Edge Middleware	Rate limiting, selective auth session refresh
Caching	Vercel Edge Cache	<code>s-maxage=3600</code> , <code>stale-while-revalidate=86400</code> on public roadmap GETs
Direct DB	pg 8.21	Admin schema application scripts

Database & auth

Layer	Technology	How you used it
Database	Supabase (Postgres)	Roadmaps, votes, community, planner (717-line schema)
Auth	Supabase Auth + @supabase/ssr	Google, GitHub, LinkedIn OAuth; cookie sessions
Security	Row Level Security (RLS)	Member-scoped planner, public-read community, maintainer-only writes
Client tiers	Anon + service role	Service role on cacheable GETs to avoid cookie cache busting
Offline-first	JSON seed files	72 roadmaps run with zero env vars for local dev and demo deploys

AI / ML pipeline

Layer	Technology	How you used it
SDK	Vercel AI SDK 6.0	Structured generation pipeline
Gateway	@ai-sdk/gateway 3.0	Provider routing via Vercel AI Gateway
Model	google/gemini-2.0-flash	Fallback parser when regex finds <3 courses
PDF extraction	unpdf 1.6	In-memory text extraction (FERPA-aware: no PDF storage)
Parsing strategy	Regex-first, AI fallback	Cost/latency control; UCSB GOLD/Banner format support
Matching	Custom normalization + graph match	Transcript course codes → roadmap node IDs

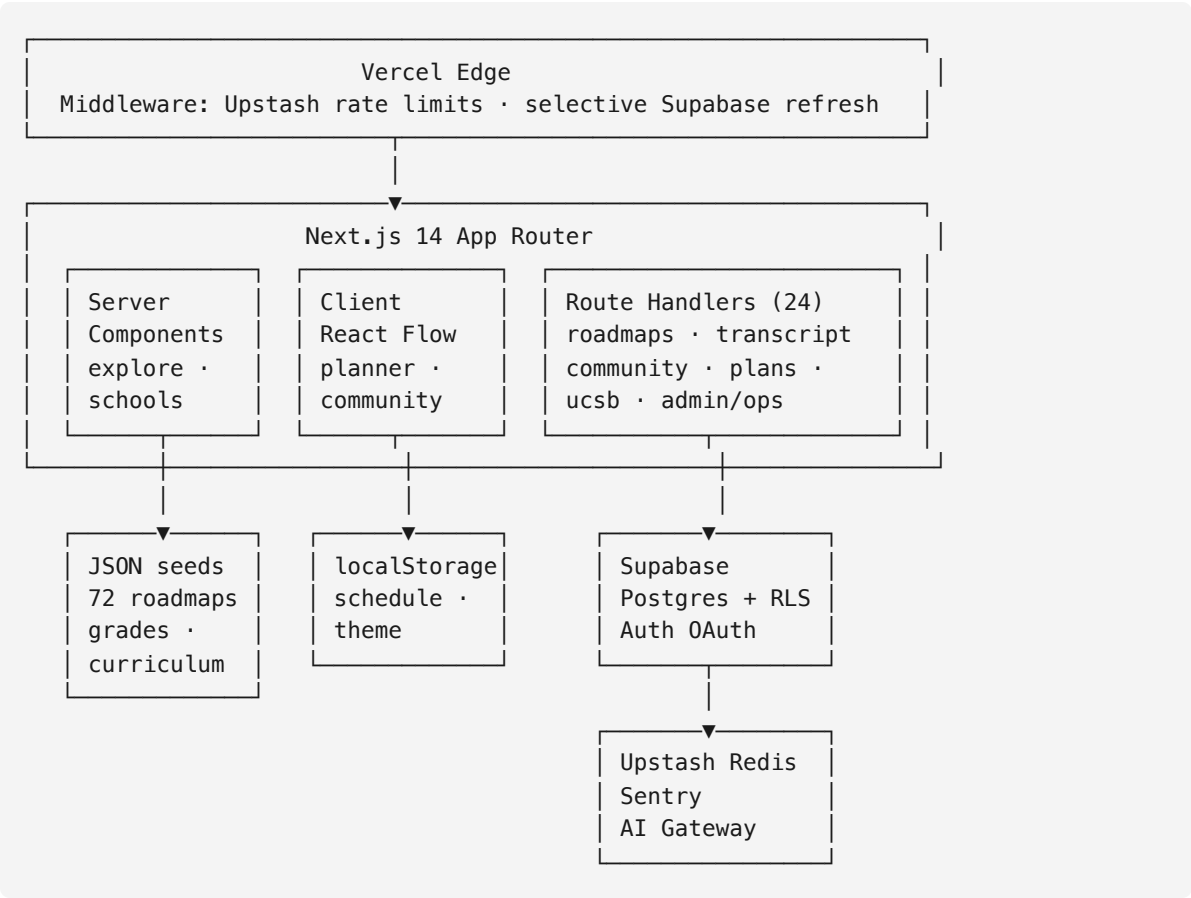
Infrastructure & production

Layer	Technology	How you used it
Hosting	Vercel	CI/CD from GitHub main, monorepo subfolder deploy
Rate limiting	Upstash Redis + @upstash/ratelimit	Sliding window: 120/min read, 10/min write, 20/hr transcript
Monitoring	Sentry 10.53	Client graph crashes, API 500s, edge middleware failures
Feature flags	Env-based	Planner collab, live UCSB connector toggles

Data engineering & tooling

Layer	Technology	How you used it
Scripts	34 TSX pipeline scripts	Fetch, validate, sync UCSB/UCLA/Berkeley catalogs, grades, faculty
Testing	Node.js test runner	Planner + transcript unit tests (14+ cases)
Linting	ESLint + eslint-config-next	CI-quality TypeScript/React

Architecture diagram



Part 2 — Feature inventory (what you actually shipped)

Use these as proof points in interviews. Numbers matter.

Feature	Scale / impact
Interactive degree roadmaps	72 seed roadmaps , 65+ UCSB, UCLA/Berkeley preview
Major requirement pages	58 L&S + 5 CoE + 9 CCS UCSB detail sheets
Explore / discovery	Search, filters, compare 2 majors, shareable URLs
Graph analysis	Prerequisite conflicts, critical path, what-if, bottlenecks
Transcript PDF import	Regex + AI fallback, UCSB GOLD format, undo last/all apply
Community hub	Q&A, reviews, alumni outcomes, announcements (Supabase RLS)
Contribute workflow	Auth-gated submissions, maintainer approval pipeline
UCSB grades explorer	Daily Nexus open data, GE area mapping, leaderboards
UCSB course catalog	Snapshot + optional live Developer API
Degree planner	8-quarter drag-drop, prereq validation, degree audit (feature-flagged)
Production hardening	Sentry, Upstash rate limits, edge cache, error boundaries

Part 3 — How to sell yourself (recruiter lens)

What FAANG recruiters scan for in 6 seconds

1. **Scope** — Did you build a toy or a system?
2. **Ownership** — End-to-end vs. "helped with UI"
3. **Production signals** — Deployed, monitored, rate-limited, tested
4. **Technical depth** — One area where you go deep (AI pipeline, caching tradeoffs, RLS)
5. **Impact language** — Users, latency, cost, reliability—not just "built X"

Your strongest positioning angles

Pick **2 primary** and **1 secondary** for each job application:

Target role	Lead with	Supporting proof
Full-stack SWE	End-to-end Next.js + Supabase + Vercel deploy	24 APIs, RLS, OAuth, 72 roadmaps
Backend SWE	API design, caching, rate limiting, Postgres RLS	Middleware split for cache hits, Upstash, admin ops
AI/ML Engineer	Hybrid regex + LLM pipeline with cost controls	Structured output, fallback logic, FERPA constraints
ML-adjacent SWE	Production AI feature, not a notebook	Transcript parse → match → apply with undo state machine

What makes this project credible (not "another todo app")

- **Dual data layer:** JSON seeds + Supabase merge—real architectural decision for zero-config dev
- **Deliberate cache/auth tradeoff:** Skip session refresh on cacheable API paths
- **FERPA-aware design:** PDF never stored, text-only AI fallback
- **Data pipeline at scale:** 34 scripts ingesting multi-campus official catalogs
- **Graph algorithms:** Critical path, bottleneck detection on prerequisite DAGs
- **Production ops docs:** You wrote runbooks, not just code

Honest gaps to address in interviews

Recruiters will probe these—have answers ready:

Gap	How to frame it
No load test metrics	"Designed for hackathon scale; rate limits and edge cache are the scaling levers I'd load-test next"
Planner collab is feature-flagged	"Architecturally complete with RLS; shipped MVP with flag for staged rollout"
AI eval is manual	"14 unit tests on parse/match/schedule; next step is golden-file eval set for transcript formats"
Solo/small team	"Owned full stack; can walk through any layer in system design"

Part 4 — STAR resume bullets

STAR format: Situation → Task → Action → Result

Resume rule: Lead with **Result**, weave in Action, keep Situation/Task implicit.

Length: 1–2 lines per bullet. Start with strong verb.

Full-stack / general SWE (pick 4–6)

1. Shipped production full-stack degree planning platform

- **S:** UC students lack a unified tool to visualize prerequisites across 70+ majors and 3 campuses.
- **T:** Build and deploy an interactive roadmap product from scratch within a hackathon timeline.
- **A:** Architected Next.js 14 App Router app with React Flow graphs, 24 REST APIs, Supabase auth/Postgres, and Vercel CI/CD; ingested official UC catalog data via 34 TypeScript pipeline scripts.
- **R:** Deployed live app covering **72 degree roadmaps** (UCSB full, UCLA/Berkeley preview) with zero-config JSON fallback for offline dev.
- **Resume bullet:** *Shipped **iGauchosBack**, a production full-stack degree roadmap platform (**72 majors**, 3 UC campuses) using **Next.js**, **Supabase**, and **Vercel**, with JSON-seed offline fallback and automated catalog ingestion pipelines.*

2. Built interactive prerequisite graph with analysis engine

- **S:** Static PDF degree sheets don't show blocking courses or graduation bottlenecks.
- **T:** Give students actionable graph analysis on live roadmap data.
- **A:** Implemented React Flow visualization with prerequisite DAG algorithms for conflict detection, critical path, what-if removal, and bottleneck scoring; persisted user schedule state in localStorage with undo-safe transcript apply tracking.
- **R:** Students can mark completed/planned courses and run what-if scenarios without backend round-trips.
- **Resume bullet:** *Built **React Flow** prerequisite graph viewer with **critical-path, conflict, and bottleneck analysis** on degree DAGs, plus persistent client-side schedule overlay with batch undo for transcript imports.*

3. Designed dual-layer data architecture (JSON + Postgres)

- **S:** Hackathon deploy couldn't require Supabase credentials; production needed auth and community writes.
- **T:** Support both demo-grade zero-config deploy and production Supabase features.
- **A:** Designed merge layer loading 72 JSON seed roadmaps first, Supabase overrides when configured; documented schema (717 lines) with RLS for community, planner, and contribution workflows.
- **R:** App runs fully functional without any env vars; production adds OAuth, submissions, and live community with one config change.
- **Resume bullet:** *Architected **dual JSON + Supabase** data layer enabling zero-env local dev while supporting **OAuth auth, RLS-scoped community writes, and maintainer approval workflows** in production.*

4. Implemented contributor submission and review pipeline

- **S:** Community-generated roadmaps need moderation before going public.
- **T:** Build auth-gated contribute flow with maintainer approval.
- **A:** Built POST/PATCH API with Supabase RLS (contributors read own pending; public reads approved only); maintainer role checks for approve/reject PATCH.
- **R:** End-to-end submission pipeline from student upload to public roadmap without service-role exposure on client.
- **Resume bullet:** *Implemented **auth-gated roadmap submission pipeline** with Supabase **RLS** policies and maintainer approve/reject workflow, separating public reads from contributor-scoped pending state.*

Backend / infrastructure (pick 3–5)

5. Production-hardened public APIs with edge caching and rate limiting

- **S:** Public roadmap APIs on Vercel faced cache-busting from auth cookies and abuse risk at launch.
- **T:** Harden GET endpoints for performance and protect write/transcript endpoints from abuse.
- **A:** Split middleware to skip Supabase session refresh on cacheable `/api/roadmaps` paths; added `Cache-Control: s-maxage=3600, stale-while-revalidate=86400`; integrated Upstash sliding-window limits (120/min read, 10/min write, 20/hr transcript); wired Sentry on API 500s and React error boundary.

- **R:** Cacheable public reads; 429 responses with Retry-After on abuse; production error visibility.
- **Resume bullet:** *Hardened production APIs with **Vercel edge caching** (1h TTL + SWR), **Upstash Redis rate limiting**, and **Sentry** observability; redesigned middleware to preserve cache hits on public GET routes.*

6. Edge middleware auth/cache split

- **S:** Supabase SSR session refresh on every request prevented CDN cache hits on public roadmap data.
 - **T:** Keep auth fresh for app pages without breaking edge cache on public APIs.
 - **A:** Implemented path-based middleware: rate-limit-only for `/api/roadmaps*` and `/api/transcript/*`; full session refresh elsewhere; service-role client on cacheable GET handlers.
 - **R:** Public roadmap list/detail responses cacheable at edge while authenticated pages stay session-fresh.
 - **Resume bullet:** *Resolved **auth vs. cache conflict** in Next.js middleware by path-splitting session refresh, enabling **edge-cached public roadmap APIs** without sacrificing OAuth session freshness on app routes.*
-

AI / ML engineering (pick 3–4)

7. Built hybrid AI transcript parsing pipeline

- **S:** Students manually mark 40+ completed courses on graphs after each enrollment period.
- **T:** Auto-import completed courses from official transcript PDFs with privacy and cost constraints.
- **A:** Built `/api/transcript/parse` pipeline: unpdf in-memory extraction → regex parser (UCSB GOLD/Banner formats) → Gemini 2.0 Flash fallback via Vercel AI Gateway with Zod structured output when regex finds <3 courses → graph node matching → preview-before-apply with undo last/all batch state machine.
- **R:** Reduced manual course marking to one PDF upload; PDF never stored (FERPA-aware); AI invoked only when regex insufficient.
- **Resume bullet:** *Designed **hybrid regex + LLM transcript parser** (Gemini via **Vercel AI Gateway**, **Zod** structured output) with **FERPA-safe in-memory PDF processing**, **auto-matching courses to prerequisite graphs**, and **batch undo state management**.*

8. Cost-aware AI fallback strategy

- **S:** Sending every transcript to an LLM is expensive and slow at scale.
- **T:** Maximize regex coverage; use AI only when necessary.
- **A:** Implemented parser selection logic triggering AI only when regex extracts fewer than 3 courses; normalized multi-format course codes (UCSB quarter prefixes, grade-before-units layout); unit-tested parse, match, and schedule apply/undo paths.
- **R:** Most text-based UCSB transcripts parse via regex alone; AI handles scanned/sparse layouts.
- **Resume bullet:** *Reduced LLM invocation cost with **regex-first parsing** and conditional AI fallback (<3 courses threshold), supporting **multi-format UC transcript layouts** with **14 automated test cases** for parse/match/schedule logic.*

Data engineering (pick 2–3)

9. Multi-campus catalog ingestion pipeline

- **S:** Official degree requirements live in PDFs and scattered web pages across UCSB, UCLA, Berkeley.
- **T:** Normalize into structured JSON seeds powering 72 interactive roadmaps.
- **A:** Built 34 TypeScript scripts to fetch, parse (GEAR grids, LASAR/DUELS, UCLA announcements), validate, and sync major detail sheets; committed snapshot JSON for curriculum, grades, and catalogs enabling deploy without runtime API keys.
- **R:** **72 validated seed roadmaps + 58 L&S major sheets** + UCSB grade distribution explorer from open data.
- **Resume bullet:** *Built **34-script TypeScript data pipeline** ingesting official **UC degree catalogs** (GEAR, LASAR, UCLA Samuelli) into validated JSON seeds powering **72 interactive roadmaps**, with snapshot-first deploy requiring no runtime API keys.*

10. UCSB grades open-data product feature

- **S:** Students lack easy access to historical grade distributions when choosing courses.
- **T:** Ship searchable grades explorer integrated with degree planning context.
- **A:** Ingested Daily Nexus open grade data; built department aggregates, GE area explorer, leaderboards, and search index; exposed via `/schools/ucsb/grades` with validation scripts.
- **R:** Full UCSB grades explorer with GE applicability mapping alongside roadmap graphs.
- **Resume bullet:** *Shipped **UCSB grade distribution explorer** by ingesting open university data into searchable aggregates with **GE area mapping** and validation pipeline, integrated into the degree planning product.*

Community / product (pick 1–2)

11. School community hub with Supabase RLS

- **S:** Students need peer Q&A, course reviews, and alumni outcomes per campus.
 - **T:** Build community features with correct auth boundaries.
 - **A:** Implemented 7 community API routes and RLS policies (public read, auth write, maintainer-only announcements); dual demo JSON / live Supabase mode; school email domain verification.
 - **R:** Per-school hub with Q&A upvotes, course review leaderboard, alumni outcomes feed.
 - **Resume bullet:** *Built **per-school community hub** (Q&A, course reviews, alumni outcomes) on **Supabase Postgres with RLS**, supporting demo JSON and production auth modes with maintainer-scoped announcement writes.*
-

Part 5 — Resume section templates

Project block (copy-paste skeleton)

iGauchoback — Full-Stack Degree Roadmap Platform

[Live Demo] [GitHub]

Next.js · TypeScript · Supabase · React Flow · Vercel AI SDK · Upstash · Sentry

https://hackathon-nu-taupe.vercel.app

- Shipped production full-stack platform covering 72 UC degree roadmaps across 3 campuses
(Next.js 14, Supabase Postgres/RLS, Vercel edge deploy)
- Built hybrid regex + LLM transcript parser (Gemini via AI Gateway) with FERPA-safe in-memory PDF processing and graph auto-matching
- Hardened public APIs with edge caching, Upstash rate limiting, and Sentry observability;
resolved auth/cache middleware conflict for CDN cache hits
- Architected dual JSON + Supabase data layer and 34-script catalog ingestion pipeline
for multi-campus official degree requirements

Skills to list (tailor per job)

Languages: TypeScript, JavaScript, SQL

Frontend: React, Next.js (App Router), Tailwind CSS, React Flow

Backend: REST API design, Node.js, Edge Middleware, Zod validation

Data: PostgreSQL, Supabase, RLS, JSON schema design, ETL scripts

AI/ML: Vercel AI SDK, LLM structured output, hybrid parsing pipelines, prompt-adjacent system design

Infra: Vercel, Upstash Redis, Sentry, CI/CD, edge caching, rate limiting

Concepts: OAuth, DAG algorithms, feature flags, snapshot-first architecture, privacy-by-design

Part 6 — Interview story bank

Prepare 2-minute versions of these:

Question	Story to tell
"Tell me about a technical tradeoff"	Auth session refresh vs. edge cache on public APIs
"Tell me about an AI feature you shipped"	Transcript regex-first + AI fallback + cost control
"Tell me about a bug you fixed in production"	UCSB GOLD transcript format (grade-before-units, quarter prefixes)
"Tell me about system design"	Dual JSON/Supabase layer, RLS tiers, planner collab schema
"Tell me about data at scale"	34-script ingestion, 72 seeds, grades snapshot pipeline
"Tell me about ownership"	Hackathon → production hardening (Sentry, Upstash, docs)

Part 7 — Portfolio presentation checklist

Must-have links

- ☐ Live demo URL on resume (above the fold)
- ☐ GitHub repo (public or accessible)
- ☐ 60-second Loom walkthrough: upload transcript → apply → undo
- ☐ README with architecture diagram (this doc helps)

GitHub polish recruiters notice

- ☐ Clear README with tech stack table and deploy link
- ☐ `docs/PRODUCTION-OPS.md` proves you think about ops
- ☐ Test scripts runnable via npm
- ☐ Meaningful commit messages (you have these)

LinkedIn post angle (one paragraph)

I built iGauchaBack because UC degree requirements are buried in PDFs. It turns official catalogs into interactive prerequisite graphs for 72 majors, lets you upload a transcript to auto-mark completed courses (regex + Gemini fallback), and ships with production hardening—edge cache, rate limits, Sentry. Stack: Next.js, Supabase, Vercel AI Gateway. Live at hackathon-nu-taupe.vercel.app

Part 8 — Role-specific tailoring guide

Applying to AI/ML roles

Lead bullets: #7, #8

Emphasize: Hybrid pipeline, structured output, cost control, eval gap + plan

Project title variant: "AI-Assisted Degree Planning Platform"

Applying to backend roles

Lead bullets: #5, #6, #4

Emphasize: API design, caching, RLS, rate limiting, middleware

Project title variant: "Full-Stack Backend for UC Degree Graph Platform"

Applying to full-stack roles

Lead bullets: #1, #2, #7, #5

Emphasize: End-to-end ownership, deploy, UX + infra

Project title variant: "iGauchaBack — Interactive UC Degree Roadmaps"

Appendix — Key metrics reference card

Metric	Value
Seed roadmaps	72
UCSB majors covered	65+ (CoE 5, L&S 58, CCS 9)
API routes	24
Page routes	18+
React components	~77
lib/ modules	~83
Data pipeline scripts	34
Unit tests	14+ (planner + transcript)
Supabase schema	~717 lines
Schools	UCSB (full), UCLA (preview), Berkeley (preview)
Production URL	hackathon-nu-taupe.vercel.app
Rate limits	120/min read · 10/min write · 20/hr transcript
Edge cache TTL	1h + 24h SWR

Document generated for portfolio and job search use. Update metrics as the project grows.